Green Bonds EDA

Noah Leibowitz & Siya Motwani

2026-06-29

```
# Load in all libraries
library(dplyr)
library(tidyverse)
library(readxl)
library(ggplot2)
library(maps)
library(tidyr)
```

```
# Solar Data
solar_2024 <- read_excel("Solar.xlsx", sheet = "2024")
solar_2023 <- read_excel("Solar.xlsx", sheet = "2023")
solar_2022 <- read_excel("Solar.xlsx", sheet = "2022")
solar_2021 <- read_excel("Solar.xlsx", sheet = "2021")
solar_2020 <- read_excel("Solar.xlsx", sheet = "2020")
solar_2019 <- read_excel("Solar.xlsx", sheet = "2019")
solar_2018 <- read_excel("Solar.xlsx", sheet = "2018")
solar_2017 <- read_excel("Solar.xlsx", sheet = "2017")
solar_2016 <- read_excel("Solar.xlsx", sheet = "2016")
solar_2015 <- read_excel("Solar.xlsx", sheet = "2015")
solar_2014 <- read_excel("Solar.xlsx", sheet = "2014")
solar_2013 <- read_excel("Solar.xlsx", sheet = "2013")

# Wind Data
wind_2024 <- read_excel("Wind.xlsx", sheet = "2024")
wind_2023 <- read_excel("Wind.xlsx", sheet = "2023")
wind_2022 <- read_excel("Wind.xlsx", sheet = "2022")
wind_2021 <- read_excel("Wind.xlsx", sheet = "2021")
wind_2020 <- read_excel("Wind.xlsx", sheet = "2020")
wind_2019 <- read_excel("Wind.xlsx", sheet = "2019")
wind_2018 <- read_excel("Wind.xlsx", sheet = "2018")
wind_2017 <- read_excel("Wind.xlsx", sheet = "2017")
wind_2016 <- read_excel("Wind.xlsx", sheet = "2016")
wind_2015 <- read_excel("Wind.xlsx", sheet = "2015")
wind_2014 <- read_excel("Wind.xlsx", sheet = "2014")
wind_2013 <- read_excel("Wind.xlsx", sheet = "2013")

# Green Bond Issuance and RPS Data
GBI <- read_excel("GB_RPS Hard Code.xlsx")

# Pre-Trends Wind Data
pre_wind <- read_csv("Wind Pre.csv")
```

```r
# Pre-Trends Solar Data
pre_solar <- read_excel("Solar 2010 - 2012.xlsx")
```

```r
# Cleaning Pre-Wind Data

clean_wind_panel <- pre_wind %>%
# Remove any rows where the operational year is missing/blank
filter(!is.na(p_year)) %>%

# Select only the specific columns your model needs
select(t_state, p_year, t_cap) %>%

# Group by State and Year, and convert individual turbine kW to total MW added
group_by(t_state, p_year) %>%
summarize(mw_added = sum(t_cap, na.rm = TRUE) / 1000, .groups = 'drop') %>%

# Open your 50-state grid for your pre-trends window
complete(t_state, p_year = 2000:2026, fill = list(mw_added = 0)) %>%
arrange(t_state, p_year) %>%
group_by(t_state) %>%

# Cumulative sum holds capacity at 0 until first project,
# then holds total capacity flat forward through the years
mutate(total_wind_mw = cumsum(mw_added)) %>%
rename(state = t_state, year = p_year) %>%

# Filter the outcome variable to strictly include 2010-2012
filter(year >= 2010 & year <= 2013)

# Split by each year and pivot year column to display state and capacity by each year separately
clean_wind_panel <- clean_wind_panel %>%
  select(state, year, total_wind_mw) %>%
  group_by(year) %>%
  group_split(.keep = FALSE)
wind_2010 <- clean_wind_panel[[1]]
wind_2011 <- clean_wind_panel[[2]]
wind_2012 <- clean_wind_panel[[3]]

wind_2013_test <- clean_wind_panel[[4]]
```

```r
# Cleaning Pre-Solar Data

# 2. Extract columns and IMMEDIATELY name them so they can't shift
solar_wide_cumulative <- pre_solar %>%
  select(
    State = 1,
    raw_2010 = 2,
    raw_2011 = 3,
    raw_2012 = 4,
    raw_2013 = 5
  )

# 3. Check column names (Optional, safe to keep)
```

```r
col_names <- colnames(solar_wide_cumulative)

# 4. Perform the math using explicit names (No indices to break!)
solar_wide_cumulative <- solar_wide_cumulative %>%
  mutate(
    # 2010 is just the base year
    Year_2010 = replace_na(raw_2010, 0),

    # 2011 = 2010 + 2011
    Year_2011 = replace_na(raw_2010, 0) + replace_na(raw_2011, 0),

    # 2012 = 2010 + 2011 + 2012
    Year_2012 = replace_na(raw_2010, 0) + replace_na(raw_2011, 0) + replace_na(raw_2012, 0),

    # 2013 = 2010 + 2011 + 2012 + 2013 (Your LBNL check anchor)
    Year_2013_test = replace_na(raw_2010, 0) + replace_na(raw_2011, 0) + replace_na(raw_2012, 0) + repla
  ) %>%

  # 5. Output only your clean State name and cumulative columns
  select(State, Year_2010, Year_2011, Year_2012, Year_2013_test)
```

```r
# Clean Post-Wind Data

# Add year columns
wind_2010 <- wind_2010 %>%
  rename(State = `state`, wind_capacity = `total_wind_mw`) %>%
  mutate(year = 2010) %>%
  arrange(State)

wind_2011 <- wind_2011 %>%
  rename(State = `state`, wind_capacity = `total_wind_mw`) %>%
  mutate(year = 2011) %>%
  arrange(State)

wind_2012 <- wind_2012 %>%
  rename(State = `state`, wind_capacity = `total_wind_mw`) %>%
  mutate(year = 2012) %>%
  arrange(State)

# Loop through every year from 2013 to 2024
for (y in 2013:2024) {

  # 1. Dynamically generate the variable name (e.g., "wind_2013")
  var_name <- paste0("wind_", y)

  # 2. Grab the dataframe from your environment using its text name
  temp_df <- get(var_name)

  # 3. Apply your tidyverse cleaning pipeline
  cleaned_df <- temp_df %>%
    mutate(year = y) %>%  # Dynamically adds the current loop's year
    rename(wind_capacity = `Nameplate Capacity (MW)`) %>%
    group_by(State, year) %>%
```

```
    summarize(wind_capacity = sum(wind_capacity, na.rm = TRUE), .groups = "drop") %>%
    arrange(State)

  # 4. Save the cleaned dataframe back to its original variable name
  assign(var_name, cleaned_df)
}

post_wind_all = bind_rows(wind_2010, wind_2011, wind_2012, wind_2013, wind_2014, wind_2015, wind_2016, w

wind_2013_uswtdb <- wind_2013_test %>%
  rename(State = state, wind_uswtdb = total_wind_mw)

wind_ratios <- wind_2013 %>%
  rename(wind_eia = wind_capacity) %>%
  left_join(wind_2013_uswtdb, by = "State") %>%
  mutate(
    # Ratio = EIA Capacity / USWTDB Capacity
    scale_factor = wind_eia / wind_uswtdb
  ) %>%
  mutate(scale_factor = if_else(is.nan(scale_factor) | is.infinite(scale_factor), 1, scale_factor)) %>%
  select(State, scale_factor)

# Scale down 2010-2012 wind capacities to meet the EIA baseline
post_wind_all <- post_wind_all %>%
  left_join(wind_ratios, by = "State") %>%
  mutate(
    wind_capacity = if_else(year <= 2012, wind_capacity * scale_factor, wind_capacity)
  ) %>%
  select(-scale_factor)

# 1. Create a master dataframe containing exactly the 48 contiguous states
contiguous_48 <- state.abb[!state.abb %in% c("AK", "HI")]
master_states <- data.frame(State = contiguous_48)

# 2. Pivot your existing combined data to wide format
wind_wide <- post_wind_all %>%
  pivot_wider(
    names_from = year,
    values_from = wind_capacity,
    names_prefix = "yr_"
  )

# 3. Left join the master list with your data and sort alphabetically
final_wind_panel <- master_states %>%
  left_join(wind_wide, by = "State") %>%
  arrange(State)

final_wind_panel_clean <- final_wind_panel %>%
  # 1. Temporarily bring columns back to a long format to work row-by-row per state
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "wind_capacity"
```

```
  ) %>%
  # 2. Group by State and sort by year to ensure sequential order
  group_by(State) %>%
  arrange(State, year) %>%
  # 3. Fill NAs with the previous year's value (Last Observation Carried Forward)
  fill(wind_capacity, .direction = "down") %>%
  # 4. If it's still NA (meaning it never did anything yet), replace with 0
  mutate(wind_capacity = replace_na(wind_capacity, 0)) %>%
  # 5. Bring it back to your beautiful wide 48-state layout
  pivot_wider(
    names_from = year,
    values_from = wind_capacity
  ) %>%
  ungroup()
```

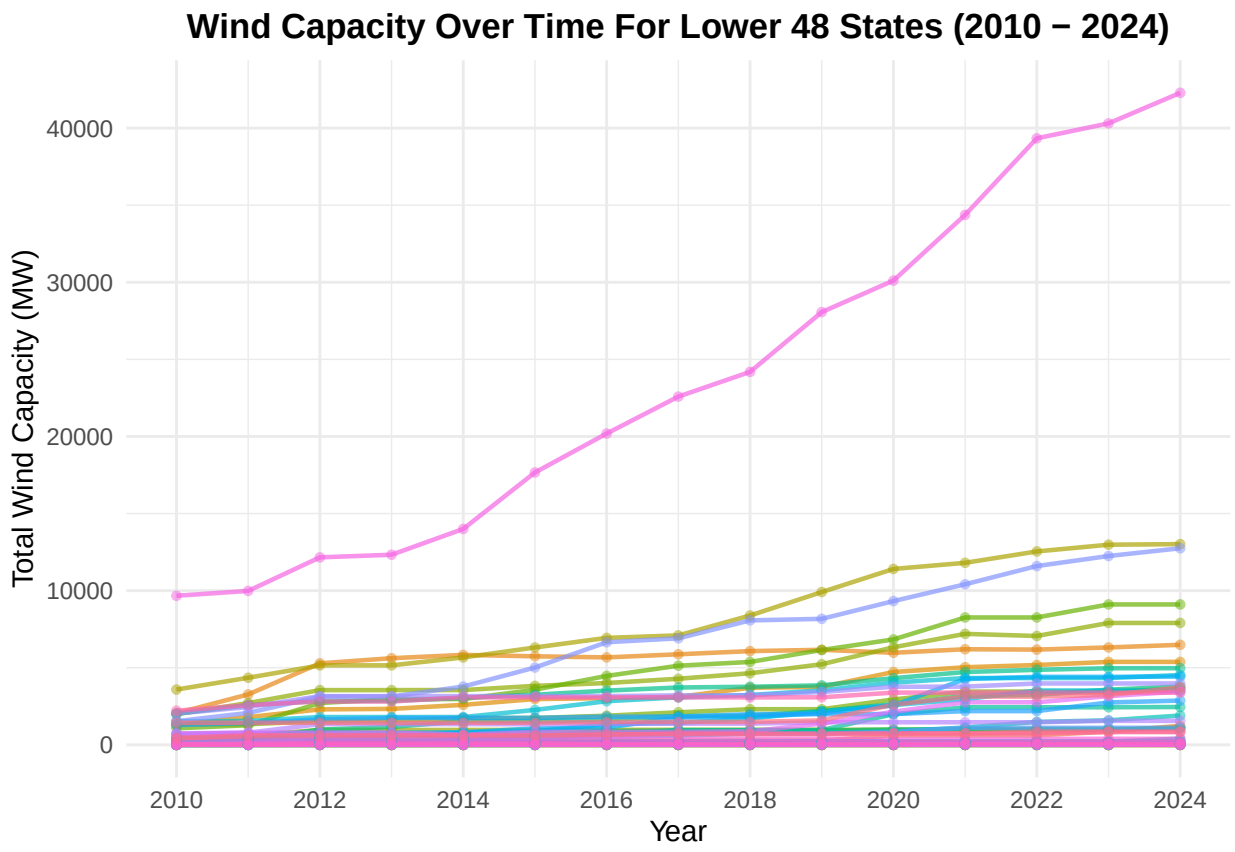
```
# Clean Post-Solar Data

# Add year columns
solar_2010 <- solar_wide_cumulative %>%
  select(State, solar_capacity = Year_2010) %>%
  mutate(
    year = 2010,
    State = state.abb[match(State, state.name)]
    ) %>%
  arrange(State)

solar_2011 <- solar_wide_cumulative %>%
  select(State, solar_capacity = Year_2011) %>%
  mutate(
    year = 2011,
    State = state.abb[match(State, state.name)]) %>%
  arrange(State)

solar_2012 <- solar_wide_cumulative %>%
  select(State, solar_capacity = Year_2012) %>%
  mutate(
    year = 2012,
    State = state.abb[match(State, state.name)]) %>%
  arrange(State)

solar_2013_test <- solar_wide_cumulative %>%
  select(State, solar_capacity = Year_2013_test) %>%
  mutate(
    year = 2013,
    State = state.abb[match(State, state.name)]) %>%
  arrange(State)

# Loop through every year from 2013 to 2024
for (y in 2013:2024) {

  # 1. Dynamically get the variable name (e.g., "solar_2013")
  var_name <- paste0("solar_", y)
```

```r
  # 2. Grab the dataframe from your environment
  temp_df <- get(var_name)

  # 3. Apply the tidyverse pipeline to sum up capacities by state
  cleaned_df <- temp_df %>%
    mutate(year = y) %>%  # Dynamically add the current loop's year
    rename(solar_capacity = `Nameplate Capacity (MW)`) %>%
    group_by(State, year) %>%
    summarize(solar_capacity = sum(solar_capacity, na.rm = TRUE), .groups = "drop") %>%
    arrange(State)

  # 4. Overwrite the messy dataframe in your environment with the cleaned one
  assign(var_name, cleaned_df)
}

post_solar_all = bind_rows(solar_2010, solar_2011, solar_2012, solar_2013, solar_2014, solar_2015, sola

# 1. Isolate the 2013 Berkeley Lab data and convert names to abbreviations
solar_2013_lbnl_check <- solar_wide_cumulative %>%
  select(State, solar_lbnl = Year_2013_test) %>%
  mutate(
    State = state.abb[match(State, state.name)]
  )

# 2. Join the EIA 2013 data with the Berkeley Lab data
solar_ratios <- solar_2013 %>%
  rename(solar_eia = solar_capacity) %>%
  left_join(solar_2013_lbnl_check, by = "State") %>%
  mutate(
    # Create a conditional scale factor to prevent division by zero or NaN errors
    scale_factor = if_else(
      is.na(solar_eia) | is.na(solar_lbnl) | solar_eia == 0 | solar_lbnl == 0,
      1,                          # Safety switch: default to 1 (do not change the data)
      solar_eia / solar_lbnl    # Otherwise, calculate the true ratio safely
    )
  ) %>%
  select(State, scale_factor)

# 3. Apply the ratio ONLY to the pre-trends years (2010-2012)
post_solar_all <- post_solar_all %>%
  left_join(solar_ratios, by = "State") %>%
  mutate(
    # Double check protection for any lingering NAs
    scale_factor = replace_na(scale_factor, 1),
    solar_capacity = if_else(year <= 2012, solar_capacity * scale_factor, solar_capacity)
  ) %>%
  select(-scale_factor)

# 1. Create a master dataframe containing exactly the 48 contiguous states
contiguous_48 <- state.abb[!state.abb %in% c("AK", "HI")]
master_states <- data.frame(State = contiguous_48)

# 2. Group and aggregate FIRST to handle hidden duplicates, then pivot wide
```

```r
solar_wide <- post_solar_all %>%
  group_by(State, year) %>%
  summarize(solar_capacity = sum(solar_capacity, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(
    names_from = year,
    values_from = solar_capacity,
    names_prefix = "yr_"
  )

# 3. Left join the master list with your solar data and sort alphabetically
final_solar_panel <- master_states %>%
  left_join(solar_wide, by = "State") %>%
  arrange(State)

# 4. Interpolate missing values cleanly
final_solar_panel_clean <- final_solar_panel %>%
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "solar_capacity"
  ) %>%
  group_by(State) %>%
  arrange(State, year) %>%
  fill(solar_capacity, .direction = "down") %>%
  mutate(solar_capacity = replace_na(solar_capacity, 0)) %>%
  pivot_wider(
    names_from = year,
    values_from = solar_capacity
  ) %>%
  ungroup()
```

```r
# 1. Transform wide data to long format for plotting
plot_data <- final_wind_panel_clean %>%
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "capacity"
  ) %>%
  # Clean up the "yr_" prefix from the year column so it treats years as numbers
  mutate(year = as.numeric(gsub("yr_", "", year)))

# 2. Create the line plot
ggplot(plot_data, aes(x = year, y = capacity, group = State, color = State)) +
  geom_line(size = 0.8, alpha = 0.7) +
  geom_point(size = 1.2, alpha = 0.5) +
  scale_x_continuous(breaks = seq(2010, 2024, by = 2)) +
  labs(
    title = "Wind Capacity Over Time For Lower 48 States (2010 - 2024)",
    x = "Year",
    y = "Total Wind Capacity (MW)"
  ) +
  theme_minimal() +
  theme(
```

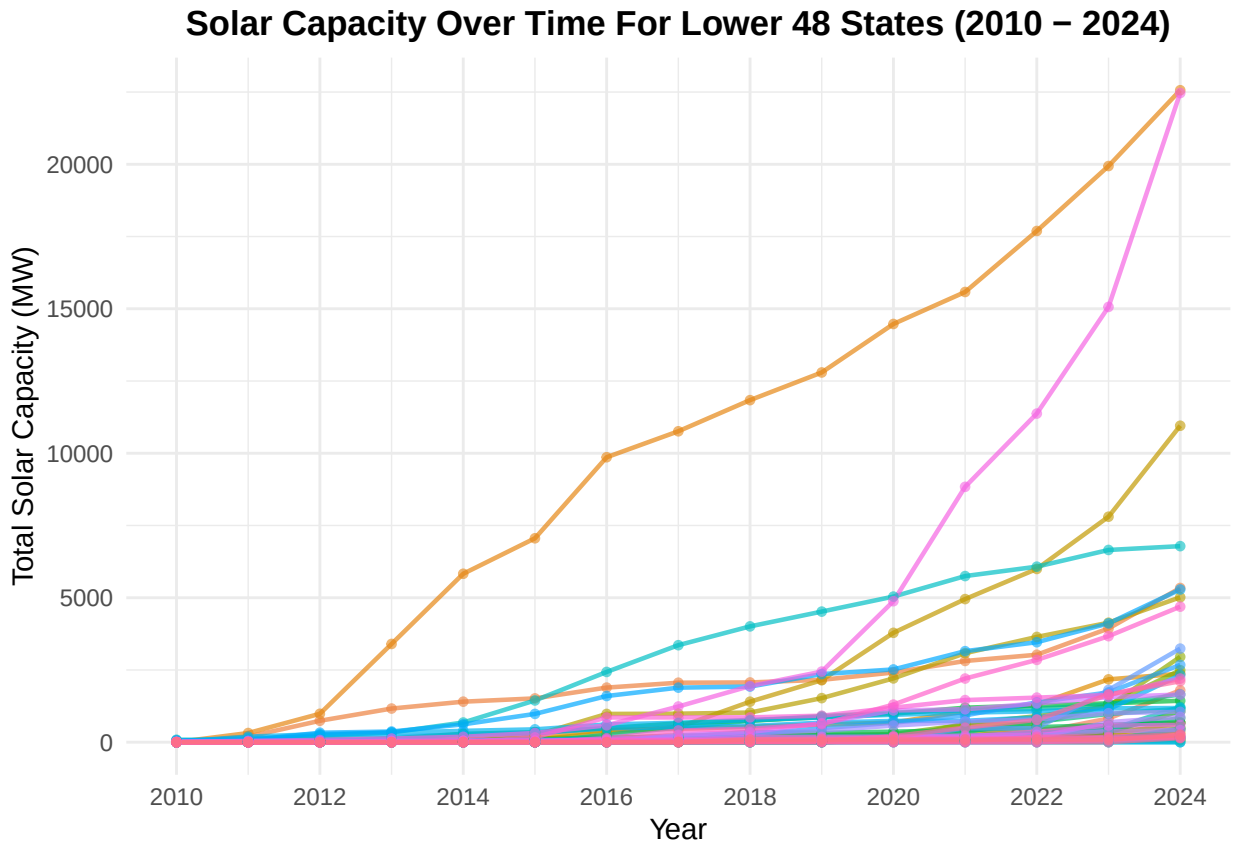
```
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5)
  )
```




```
ggsave(
  filename = "wind_capacity_timeline.png",
  width = 10,        # Width in inches
  height = 6,        # Height in inches
  dpi = 300          # Publication-quality resolution (300 dots-per-inch)
)
```

```
# 1. Transform wide data to long format for all years
solar_plot_data_all <- final_solar_panel_clean %>%
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "capacity"
  ) %>%
  # Clean up the "yr_" prefix and turn year into a number
  mutate(year = as.numeric(gsub("yr_", "", year)))

# 2. Create the full-timeline line plot
ggplot(solar_plot_data_all, aes(x = year, y = capacity, group = State, color = State)) +
  geom_line(size = 0.8, alpha = 0.7) +
```

```
  geom_point(size = 1.2, alpha = 0.6) +
  # Ensure the X-axis handles the wide timeframe cleanly
  scale_x_continuous(breaks = seq(2010, 2024, by = 2)) +
  labs(
    title = "Solar Capacity Over Time For Lower 48 States (2010 - 2024)",
    x = "Year",
    y = "Total Solar Capacity (MW)"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5)
  )
```




```
ggsave(
  filename = "solar_capacity_timeline.png",
  width = 10,        # Width in inches
  height = 6,        # Height in inches
  dpi = 300          # Publication-quality resolution (300 dots-per-inch)
)
```

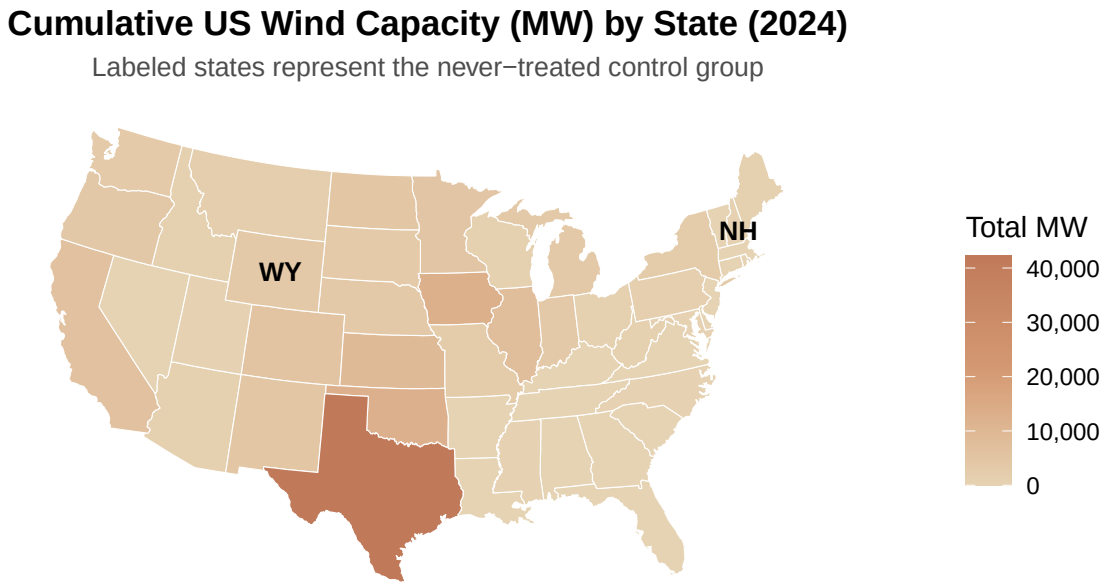
```
# Building the Panel Dataset

# Create a dedicated long-format version for regressions and plotting
```

```r
final_wind_panel_long <- final_wind_panel %>%
  # 1. Pivot permanently to long format
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "wind_capacity"
  ) %>%
  # 2. Strip "yr_" so the year column becomes a clean, parseable number
  mutate(year = as.numeric(gsub("yr_", "", year))) %>%
  # 3. Sort sequentially by state and year to prepare for the carry-forward step
  group_by(State) %>%
  arrange(State, year) %>%
  # 4. Handle missing data gaps using Last Observation Carried Forward (LOCF)
  fill(wind_capacity, .direction = "down") %>%
  # 5. Switch any remaining early NAs to 0
  mutate(wind_capacity = replace_na(wind_capacity, 0)) %>%
  # 6. Ungroup so it behaves like a standard dataframe for downstream packages
  ungroup()

# Create a dedicated long-format version for solar regressions and plotting
final_solar_panel_long <- final_solar_panel %>%
  # 1. Pivot permanently to long format
  pivot_longer(
    cols = starts_with("yr_"),
    names_to = "year",
    values_to = "solar_capacity"
  ) %>%
  # 2. Strip "yr_" so the year column becomes a clean, parseable number
  mutate(year = as.numeric(gsub("yr_", "", year))) %>%
  # 3. Sort sequentially by state and year to prepare for the carry-forward step
  group_by(State) %>%
  arrange(State, year) %>%
  # 4. Handle missing data gaps using Last Observation Carried Forward (LOCF)
  fill(solar_capacity, .direction = "down") %>%
  # 5. Switch any remaining early NAs to 0
  mutate(solar_capacity = replace_na(solar_capacity, 0)) %>%
  # 6. Ungroup so it behaves nicely for downstream models
  ungroup()

# One-hot-encoding of corporate & munucipal issuances, and RPS Issuances
# Corporate: 1 if current year >= corporate year, otherwise 0
# Municipal: 1 if current year >= municipal year, otherwise 0
# RPS: 1 if current year >= RPS enacted AND current year < RPS expired. If never expired, remains 1
# GB: 1 if a state issued either type of green bond, 0 if a state issued neither
policy_clean <- GBI %>%
  # 1. Standardize state names to postal abbreviations
  mutate(State = state.abb[match(State, state.name)]) %>%
  filter(!is.na(State)) %>%

  # 2. Convert character strings/NAs to proper numeric years or infinity
  mutate(
    corp_start  = as.numeric(gsub("N/A", NA, `Corporate Year`)),
    muni_start  = as.numeric(gsub("N/A", NA, `Municipal Year`)),
```

```
    rps_start   = as.numeric(gsub("N/A", NA, `RPS Year`)),
    rps_end     = as.numeric(gsub("N/A", NA, `RPS Repealed/Expired`)),

    # If RPS was never repealed, set its expiration to infinity so it stays active
    rps_end = if_else(is.na(rps_end), Inf, rps_end)
  ) %>%
  select(State, corp_start, muni_start, rps_start, rps_end)

# 2. Merge Wind and Solar long panels into a single JOINT panel
joint_energy_panel <- final_wind_panel_long %>%
  full_join(final_solar_panel_long, by = c("State", "year"))

# Combine panels and compute the logical thresholds, including the combined 'gb' column
final_joint_regression_panel <- final_wind_panel_long %>%
  # Join wind and solar together side-by-side
  full_join(final_solar_panel_long, by = c("State", "year")) %>%

  # Join your cleaned timeline parameters
  left_join(policy_clean, by = "State") %>%

  # Build the binary metrics dynamically
  mutate(
    is_corporate = if_else(!is.na(corp_start) & year >= corp_start, 1, 0),
    is_public    = if_else(!is.na(muni_start) & year >= muni_start, 1, 0),
    gb           = if_else(is_corporate == 1 | is_public == 1, 1, 0),

    # RPS Policy Dummy
    is_rps       = if_else(!is.na(rps_start) & year >= rps_start & year < rps_end, 1, 0)
  ) %>%

  # Clean up helper metrics
  select(-corp_start, -muni_start, -rps_start, -rps_end)

# Siya, once we add the rest of the variables to this panel, we'll export it as a csv
# Add and clean the rest of the variables below:

# Export final panel dataset to a CSV file
write.csv(final_joint_regression_panel, "panel.csv", row.names = FALSE)
```

```
# Wind map

# 1. Determine which states are Never Treated (gb == 0 globally)
never_treated_states <- final_joint_regression_panel %>%
  group_by(State) %>%
  summarize(ever_issued = max(gb, na.rm = TRUE), .groups = "drop") %>%
  filter(ever_issued == 0) %>%
  pull(State)

# 2. Prepare 2024 data
wind_map_2024 <- final_wind_panel_clean %>%
  select(State, capacity = yr_2024) %>%
  mutate(region = tolower(state.name[match(State, state.abb)]))
```

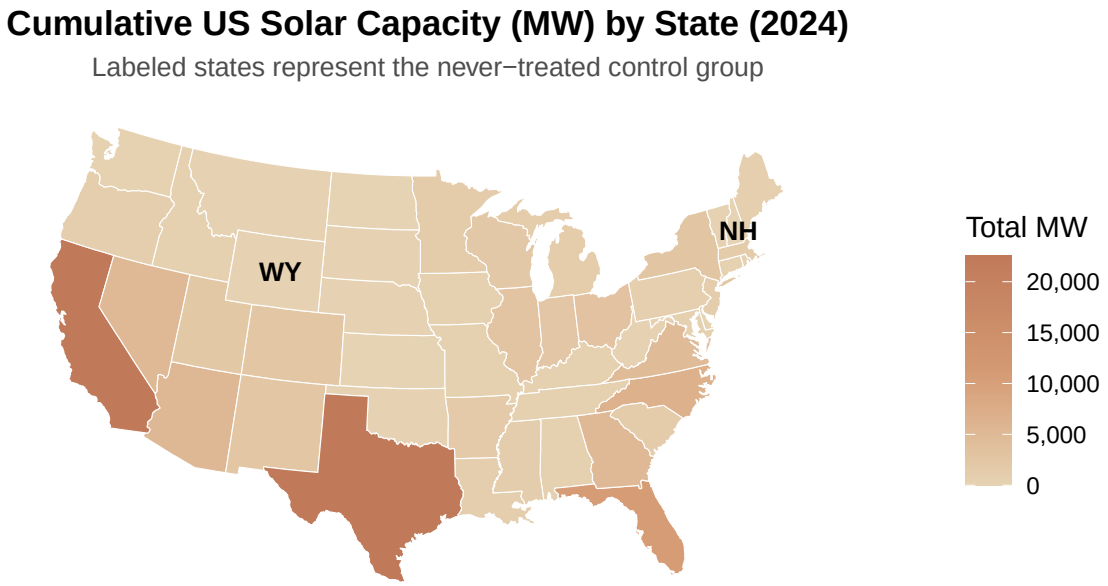
```r
us_states <- map_data("state")

wind_geo_data <- us_states %>%
  left_join(wind_map_2024, by = "region")

# 3. Create a label dataset containing ONLY the never-treated states
never_treated_labels <- data.frame(
  State = state.abb,
  long = state.center$x,
  lat = state.center$y
) %>%
  # Filter out AK and HI, and isolate ONLY the never-treated control states
  filter(!State %in% c("AK", "HI"), State %in% never_treated_states)

# 4. Generate Wind Map
ggplot() +
  # Base map layer showing capacities
  geom_polygon(data = wind_geo_data, aes(x = long, y = lat, group = group, fill = capacity),
               color = "white", size = 0.2) +

  # Overlay text labels ONLY for the never-treated states
  geom_text(data = never_treated_labels, aes(x = long, y = lat, label = State),
            color = "black", fontface = "bold", size = 3.5, check_overlap = TRUE) +

  # Custom continuous gradient interpolation using your colors
  scale_fill_gradientn(
    name = "Total MW",
    colors = c("#E6D3B3", "#D49A73", "#C07A5A"),
    labels = scales::comma_format(),
    na.value = "grey90"
  ) +
  coord_map("albers", lat0 = 39, lat1 = 45) +
  labs(
    title = "Cumulative US Wind Capacity (MW) by State (2024)",
    subtitle = "Labeled states represent the never-treated control group"
  ) +
  theme_void() +
  theme(
    legend.position = "right",
    legend.box.margin = margin(t = 0, r = 20, b = 0, l = 0),
    legend.margin = margin(t = 0, r = 0, b = 0, l = 0),
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 10, color = "gray30")
  )
```




```
ggsave(
  filename = "wind_map.png",
  width = 10,        # Width in inches
  height = 6,        # Height in inches
  dpi = 300          # Publication-quality resolution (300 dots-per-inch)
)
```

```
# Solar map

# 1. Determine which states are Never Treated (gb == 0 globally)
never_treated_states <- final_joint_regression_panel %>%
  group_by(State) %>%
  summarize(ever_issued = max(gb, na.rm = TRUE), .groups = "drop") %>%
  filter(ever_issued == 0) %>%
  pull(State)

# 2. Prepare 2024 Solar data
solar_map_2024 <- final_solar_panel_clean %>%
  select(State, capacity = yr_2024) %>%
  mutate(region = tolower(state.name[match(State, state.abb)]))

us_states <- map_data("state")

solar_geo_data <- us_states %>%
  left_join(solar_map_2024, by = "region")
```

```r
# 3. Create a label dataset containing ONLY the never-treated states
never_treated_labels <- data.frame(
  State = state.abb,
  long = state.center$x,
  lat = state.center$y
) %>%
  # Filter out AK and HI, and isolate ONLY the never-treated control states
  filter(!State %in% c("AK", "HI"), State %in% never_treated_states)

# 4. Generate Solar Map
ggplot() +
  # Base map layer showing solar capacities
  geom_polygon(data = solar_geo_data, aes(x = long, y = lat, group = group, fill = capacity),
               color = "white", size = 0.2) +

  # Overlay text labels ONLY for the never-treated states
  geom_text(data = never_treated_labels, aes(x = long, y = lat, label = State),
            color = "black", fontface = "bold", size = 3.5, check_overlap = TRUE) +

  # Custom continuous gradient interpolation using your colors
  scale_fill_gradientn(
    name = "Total MW",
    colors = c("#E6D3B3", "#D49A73", "#C07A5A"),
    labels = scales::comma_format(),
    na.value = "grey90"
  ) +
  coord_map("albers", lat0 = 39, lat1 = 45) +
  labs(
    title = "Cumulative US Solar Capacity (MW) by State (2024)",
    subtitle = "Labeled states represent the never-treated control group"
  ) +
  theme_void() +
  theme(
    legend.position = "right",
    legend.box.margin = margin(t = 0, r = 20, b = 0, l = 0),
    legend.margin = margin(t = 0, r = 0, b = 0, l = 0),
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 10, color = "gray30")
  )
```




```
ggsave(
  filename = "solar_map.png",
  width = 10,        # Width in inches
  height = 6,        # Height in inches
  dpi = 300          # Publication-quality resolution (300 dots-per-inch)
)
```